Technical Blocker Mitigation — Defender for Cloud

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Technical Blocker Mitigation: Defender for Cloud 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Boundary . . . 4

Effort . . . 5

1. Solution Overview — Technical Blocker Mitigation: Defender for Cloud 5

What this solution is . . . 5

How Defender for Cloud signal actually works (so you can find the break) . . . 5

The standard deployment model (applied to a blocker) . . . 6

Scope . . . 6

Out of scope . . . 6

Connections and authentication used . . . 7

2. Requirements Guide — Technical Blocker Mitigation: Defender for Cloud 7

2.1 Read-only access to the affected subscriptions and connectors . . . 7

2.2 Az PowerShell modules available . . . 7

2.3 The affected scope defined . . . 7

2.4 Log Analytics workspace reachable (where used) . . . 8

2.5 Multicloud connector onboarding artifacts known (AWS/GCP) . . . 8

2.6 A place to store the diagnostic output and support bundle . . . 8

Checklist . . . 8

Global Requirements — Defender (category-level) 9

G1. Microsoft 365 / Entra tenant . . . 9

G2. Defender licensing . . . 9

G3. Administrative roles (least privilege) . . . 9

G4. Onboarding reach to endpoints / identity / workloads . . . 9

G5. Network access to Defender service endpoints . . . 9

G6. A pilot ring . . . 10

G7. Log Analytics workspace (where required) . . . 10

G8. Response governance . . . 10

3. Implementation Guide — Technical Blocker Mitigation: Defender for Cloud 10

Naming (set once, reuse verbatim) . . . 10

Step 1 — Assess: pinpoint the blocker (read-only) . . . 10

Step 2 — Assess: analyze agent / agentless coverage (read-only) . . . 11

Step 3 — Configure: apply the one targeted fix (manual, documented, change-approved) . . . 11

Step 4 — Pilot: re-test the fix on the affected scope only . . . 12
Step 5 — Validate: confirm the blocker is cleared . . . 12
Step 6 — Expand: resume the stalled rollout . . . 12
Step 7 — Escalate (only if it is a service issue, not your config) . . . 12
5. Testing Guide — Technical Blocker Mitigation: Defender for Cloud 13
Test 1 — Blocker pinpointed (baseline captured) . . . 13
Test 2 — Defender plan enabled . . . 13
Test 3 — Agent / agentless coverage . . . 13
Test 4 — AWS / GCP connector connected . . . 13
Test 5 — Recommendations / assessments populate . . . 13
Test 6 — Policy assignment present . . . 13
Test 7 — Before/after evidence produced . . . 14
Test 8 — Escalation bundle valid (only if escalating) . . . 14
6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Cloud 14
Plan will not enable / reverts to Free . . . 14
Agent / agentless coverage gap . . . 14
AWS/GCP connector failure . . . 15
Recommendations not populating . . . 15
Script cannot authenticate / read . . . 15
When to escalate to Microsoft support . . . 15
7. Best Practices — Technical Blocker Mitigation: Defender for Cloud 15
Diagnose before you enable anything paid . . . 16
Change one thing, on the affected scope, change-approved, then verify . . . 16
Watch for plans that silently revert . . . 16
Coverage: know agent vs agentless . . . 16
Multicloud is a permissions problem until proven otherwise . . . 16
Let plans, coverage, connectors, and assessments warm up . . . 16
Keep licensing, plan, and cost facts current . . . 16
Escalate with evidence, not a description . . . 16
Where to go next . . . 17
8. Deliverables Summary — Technical Blocker Mitigation: Defender for Cloud 17
Documentation . . . 17
PowerShell scripts (Az PowerShell — Az.Security / Az.Resources — all read-only) . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Technical Blocker Mitigation: Defender for Cloud

White-label · Defender

A Microsoft Defender for Cloud rollout almost always stalls on the same short list of blockers: a **plan that will not enable** (or silently reverts to Free), **agent / agentless coverage gaps** (machines the plan is paying for that report no data), **AWS / GCP connector failures** (a multicloud connector that never finishes onboarding or stops syncing), or **recommendations that never populate** (Secure Score and assessments stay empty because the policy/assessment layer is not flowing). Left undiagnosed, the org pays for coverage it is not getting. This solution is a **diagnostic-first playbook**: read-only discovery that pinpoints *which* blocker you actually have, the **targeted, documented manual fix**, and the verification that proves the plan, coverage, connector, or recommendations are working — so a stuck rollout gets moving again fast.

This is a **run-it-yourself** package written for an administrator with no prior deep Defender for Cloud troubleshooting experience. Every script is **read-only** — it reads plan state, coverage, connector status, and assessment data; it never changes a plan, an agent setting, a connector, or a policy. Any fix is a **documented manual step** you apply deliberately, on the affected subscription / connector subset first.

What you get

- A plain-language **Solution Overview** of the Defender for Cloud signal chain — plan enablement, agent/agentless coverage, multicloud connectors, and the policy/assessment layer — and where each one breaks.
- A **Requirements Guide** with verify / implement-if-missing for the diagnostic-specific prerequisites.
- An **Implementation Guide** that walks the diagnostic loop end to end — pinpoint the blocker, apply the targeted fix, re-test on the affected scope, confirm working, resume the rollout.
- Five **read-only PowerShell scripts** (Az PowerShell — Az.Security / Az.Resources) for discovery, coverage analysis, blocker validation, a status report, and a support-ready data bundle.
- **Testing** and **Troubleshooting** guides that map every common symptom to its cause and fix.

What this delivers

A previously stuck Defender for Cloud rollout **moving again** — the blocker diagnosed, the targeted fix applied and verified on the affected scope, and, if the root cause is a Microsoft service issue rather than your configuration, a **support-ready diagnostic bundle** and clear escalation criteria.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess with read-only discovery to pinpoint the blocker, apply the targeted fix, pilot the fix on the affected subscription / connector, validate the plan / coverage / connector / recommendations work, then expand by resuming the stalled rollout.

Boundary

This is a **diagnostic and troubleshooting** solution. It gets a *stalled* Defender for Cloud deployment unstuck; it does **not** replace a full CSPM / Secure Score remediation engagement (see the adjacent *Defender for Cloud — CSPM & Secure Score* solution). All scripts are **read-only**. Enabling paid plans, granting cloud roles, and remediating recommendations stay **customer-owned and change-approved**; incident-response and licensing/cost determinations remain the customer's functions — this guide provides the technical diagnosis and the documented fix, not the decision.

Effort

A competent administrator can run discovery, identify the blocker, apply the documented fix, and verify the plan / coverage / connector / recommendations for a single blocker class in a focused session. Rolling the fix across every subscription and multicloud connector scales with the size of the estate.

1. Solution Overview — Technical Blocker Mitigation: Defender for Cloud

What this solution is

This is a **diagnostic and troubleshooting playbook** for a **stalled Microsoft Defender for Cloud rollout**. It assumes you have started enabling plans, provisioning agents, or onboarding AWS/GCP, and hit a wall — a plan that will not stay on, machines with no data, a multicloud connector that never syncs, or an empty Secure Score. Its job is to answer three questions quickly and in order:

1. **Which blocker do I actually have?** (read-only discovery of plan, coverage, connector, and assessment state — not guesswork)
2. **What is the exact, documented fix?** (a targeted manual step, applied deliberately)
3. **Is the plan / coverage / connector / recommendations now working?** (verification before you resume)

It targets the four blocker classes that stall almost every Defender for Cloud deployment:

- **Plan not enabling** — a Defender plan (Servers, Storage, Containers, Defender CSPM, etc.) will not turn on for a subscription, or silently reverts to **Free**, so paid protections never apply.
- **Agent / agentless coverage gaps** — machines the plan covers report **no data** because the agent (Azure Monitor Agent / the Defender extension) is not provisioned, or **agentless scanning** is not enabled, leaving blind spots.
- **AWS / GCP connector failures** — a **multicloud connector** never finishes onboarding (the CloudFormation / Terraform role or permissions are wrong) or stops syncing, so AWS/GCP resources are unassessed.
- **Recommendations not populating** — **Secure Score** and assessments stay empty because the **Microsoft cloud security benchmark** policy assignment is missing/failing or assessment data is not flowing.

How Defender for Cloud signal actually works (so you can find the break)

Defender for Cloud has a signal chain, and a blocker is always a break in one specific link:

1. **Plan enablement** — the required **Defender plans** are set to **On** (Standard) at the subscription (or connector) scope. *Verify current plan names/pricing in your tenant.*
2. **Coverage** — covered resources are instrumented: **agents** (auto-provisioning of the Azure Monitor Agent / Defender extension) and/or **agentless scanning** are enabled so machines and services actually report.
3. **Multicloud connectors** — **AWS/GCP connectors** are onboarded with the correct roles/permissions and are **syncing**, so non-Azure resources are in scope.
4. **Policy / assessment layer** — the **Microsoft cloud security benchmark** (Azure Policy) is assigned and evaluating, feeding **assessments** and **Secure Score**.
5. **Verification** — plans show **On**, coverage shows machines reporting, connectors show **connected**, and assessments/recommendations populate with a Secure Score you can trend.

Find the earliest broken link and you have found the blocker. Discovery reports the state of each link so you are not guessing.

The standard deployment model (applied to a blocker)

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand. Assess*** *— run read-only discovery and coverage analysis to pinpoint the earliest broken link.* ***Configure*** *— apply the single documented fix for that blocker (a manual portal/CLI step, change-approved, not a script action).* ***Pilot*** *— re-test on the* ***affected subscription / connector*** *only.* ***Validate*** *— confirm the plan / coverage / connector / recommendations work.* ***Expand*** *— resume the stalled rollout across the remaining subscriptions and connectors.*

Scope

- Diagnosing and clearing the four blocker classes above for a stalled Defender for Cloud rollout.
- Read-only discovery, agent/agentless coverage analysis, validation, status reporting, and a support-ready bundle.
- Documented manual fixes for each blocker, applied affected-scope-first and change-approved.
- Clear escalation criteria and the exact data to collect when the root cause is a Microsoft service issue.

Out of scope

- **A full CSPM / Secure Score remediation engagement** — that is the adjacent *Defender for Cloud — CSPM & Secure Score* solution; this one unblocks a rollout already in progress.
- **Remediating individual recommendations** — clearing findings and hardening resources is the customer's change-approved work; here recommendations are diagnosed only for whether they *populate*.
- **Cost / plan-selection decisions** — enabling paid plans has cost impact; that decision is the customer's finance/ops function.

• **Licensing determinations** — the guide tells you *which* plan a capability needs and how to verify it; enabling it is the customer's decision.

Connections and authentication used

• **Microsoft Defender for Cloud** in the Azure portal (`portal.azure.com` → Defender for Cloud) — plans, coverage, connectors, recommendations, Secure Score.

• **Az PowerShell** (`Az.Accounts`, `Az.Security`, `Az.Resources`, `Az.PolicyInsights`) for read-only discovery, coverage, validation, and reporting; ARM REST reads for multicloud connectors.

• **AWS / GCP** onboarding artifacts (CloudFormation / Terraform roles) as the connector trust — read-only status checks here.

• Least-privilege roles: **Security Reader** / **Reader** on the subscriptions for all read-only diagnostics; **Security Admin** / **Owner-delegated** roles only to apply a fix (enable a plan, provisioning, or a connector).

2. Requirements Guide — Technical Blocker Mitigation: Defender for Cloud

Verify **every** requirement before diagnosing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, Log Analytics workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running the Defender for Cloud blocker diagnostics**. Objects you create are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Read-only access to the affected subscriptions and connectors

Description: Security Reader or **Reader** on the Azure subscriptions (and read access to the security connectors) in scope, plus Az PowerShell signed in to the correct tenant. **Why:** The diagnostics read plan state, coverage, connector status, and assessments across the subscriptions — all read-only — to pinpoint the blocker. **Verify:** `Connect-AzAccount` succeeds; `Get-AzSecurityPricing` returns plan states for a subscription. `TBMMDC-Discovery.ps1` reports the subscriptions and roles it resolved. **Implement-if-missing:** Assign **Security Reader / Reader** at the subscription (or management-group) scope.

2.2 Az PowerShell modules available

Description: The **Az.Accounts**, **Az.Security**, **Az.Resources**, and **Az.PolicyInsights** modules on the workstation running the diagnostics. **Why:** Plan state, auto-provisioning, assessments, and policy compliance are exposed through Az cmdlets and ARM REST. **Verify:** `Get-Module -ListAvailable Az.Security` returns a current version; `TBMMDC-Discovery.ps1` installs Az if missing. **Implement-if-missing:** `Install-Module Az -Scope CurrentUser` (or at least the modules above).

2.3 The affected scope defined

Description: A named subset of the subscriptions / connectors exhibiting the blocker — the ones you will re-test the fix against before resuming. Name it `MDC Blocker Triage – Affected Scope – Diagnose`. **Why:** Fixes are validated on the affected scope first (per `global-requirements-defender.md` G6) before touching the whole estate. **Verify:** The list of affected subscription IDs / connector names exists; `TBMMDC-Validate.ps1` checks their state. **Implement-if-missing:** Document the affected subscriptions/connectors as `MDC Blocker Triage – Affected Scope – Diagnose`.

2.4 Log Analytics workspace reachable (where used)

Description: The **Log Analytics workspace** Defender for Cloud is configured to use (for agent-based data / continuous export), reachable and access-controlled. **Why:** Coverage gaps often trace to a workspace the agent cannot reach or is not linked to; you must be able to read the workspace/agent linkage. **Verify:** The workspace exists and the affected machines' agents point at it; `TBMMDC-Coverage.ps1` reports agent/workspace linkage hints. (See `global-requirements-defender.md` G7.) **Implement-if-missing:** Confirm/relink the workspace per current Defender for Cloud guidance for the fix step.

2.5 Multicloud connector onboarding artifacts known (AWS/GCP)

Description: For AWS/GCP blockers, know how the connector was onboarded — the **CloudFormation** (AWS) or **Terraform / gcloud** (GCP) role/permissions the connector uses. **Why:** Connector failures are almost always a role/permission problem; you must know the onboarding method to diagnose and re-run it. **Verify:** `TBMMDC-Discovery.ps1` lists security connectors and their state; you can view the AWS role / GCP service account used. *Verify current multicloud onboarding requirements.* **Implement-if-missing:** Locate the connector onboarding artifacts (or plan to re-run onboarding) with whoever owns the AWS/GCP account.

2.6 A place to store the diagnostic output and support bundle

Description: A secure location for the JSON snapshots the scripts write and for the support bundle `MDC Support Bundle – <case> – Escalate`. **Why:** The before/after snapshots prove the blocker cleared, and the bundle is what you attach to a Microsoft support case. **Verify:** A working folder exists with least-privilege access; `TBMMDC-Report.ps1` writes there. **Implement-if-missing:** Create a restricted working folder; do not store subscription IDs / cloud-account identifiers in shared/unsecured locations.

Checklist

- Security Reader / Reader on the affected subscriptions + Az signed in — `TBMMDC-Discovery.ps1`
- Az modules (Az.Accounts / Az.Security / Az.Resources / Az.PolicyInsights) installed
- Affected scope `MDC Blocker Triage – Affected Scope – Diagnose` documented
- Log Analytics workspace reachable / linkage known — `TBMMDC-Coverage.ps1`
- Multicloud connector onboarding artifacts known (AWS CloudFormation / GCP Terraform)
- Secure working folder for snapshots and the support bundle

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Technical Blocker Mitigation: Defender for Cloud

Follow these steps **in order**. This is a **diagnostic loop**, not a build: you assess to pinpoint the blocker, apply one targeted documented fix (change-approved, since enabling paid plans has cost impact), re-test the affected scope, and only then resume the rollout. All scripts are **read-only**; every *fix* is a manual step you apply deliberately. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assess (pinpoint) → Configure (fix) → Pilot (re-test affected scope) → Validate (confirm working) → Expand (resume rollout). Enabling paid plans and remediation stay change-approved and customer-owned throughout.*

Naming (set once, reuse verbatim)

- Affected subscription/connector scope: `MDC Blocker Triage – Affected Scope – Diagnose`
- Support bundle output: `MDC Support Bundle – <case> – Escalate`
- Discovery snapshot file: `MDC-blocker-discovery.json`
- Coverage snapshot file: `MDC-coverage.json`

Step 1 — Assess: pinpoint the blocker (read-only)

1. Sign in with `Connect-AzAccount` to the correct tenant as **Security Reader / Reader**.

2. Run `TBMMDC-Discovery.ps1`. It captures the signal chain per subscription in `MDC-blocker-discovery.json`: **Defender plan states** (`Get-AzSecurityPricing` — On/Free per plan), **auto-provisioning / agent** settings, **security connectors** (AWS/GCP) and their status, and whether the **Microsoft cloud security benchmark** policy is assigned and assessments exist.

3. Assemble `MDC Blocker Triage – Affected Scope – Diagnose` from the subscriptions/connectors showing the symptom.

4. Read the snapshot top-down against the signal chain (Overview §"How signal actually works") and identify the **earliest broken link**:

• Plan shows Free / will not enable → **plan-enablement blocker**

• Plan On but machines report no data → **coverage blocker**

• Connector not connected / not syncing → **AWS/GCP connector blocker**

• Coverage fine but Secure Score / assessments empty → **policy/assessment blocker**

Step 2 — Assess: analyze agent / agentless coverage (read-only)

1. Run `TBMMDC-Coverage.ps1`. It reports, for the affected scope, which machines are covered by an agent/extension vs agentless scanning, and flags **coverage gaps** (machines a paid plan covers that report no data).

2. Note whether the gap is an **agent provisioning** problem (auto-provisioning off / workspace unreachable) or an **agentless** enablement problem — they have different fixes.

Step 3 — Configure: apply the one targeted fix (manual, documented, change-approved)

Apply **only** the fix for the blocker you identified. Each is a deliberate, change-approved manual step — the scripts never make these changes.

• **Plan not enabling:** In **Defender for Cloud** → **Environment settings** → **[subscription]** → **Defender plans**, set the required plan to **On** and save. If it reverts, check for an **Azure Policy / deny** or a management-group setting overriding it, and the registering **resource providers**. *Enabling a paid plan has cost impact — change-approved.* *Verify current plan names/pricing.*

• **Agent / agentless coverage gap:** Turn on **auto-provisioning** of the required agent/extension (or fix the workspace linkage the agent points at), or enable **agentless scanning** in the plan settings. Confirm the affected machines pick up the extension. (See `global-requirements-defender.md` G7 for the workspace.)

• **AWS / GCP connector failure:** In **Environment settings** → **Add/select the connector**, re-run onboarding — correct the **CloudFormation** (AWS) role/permissions or the **Terraform / gcloud** service account (GCP), and confirm the connector returns to **connected / syncing**. *Verify current multicloud onboarding requirements.*

• **Recommendations not populating:** Confirm the **Microsoft cloud security benchmark** policy assignment exists and is evaluating at the affected scope; trigger/await a compliance evaluation. Confirm the registering resource providers and that assessments have had time to populate.

Step 4 — Pilot: re-test the fix on the affected scope only

1. Apply the fix to the subscription/connector in `MDC Blocker Triage – Affected Scope – Diagnose` only — not the whole estate.
2. Give plan enablement, provisioning, connector sync, and assessment evaluation time (these populate over hours).
3. Move to validation before touching any other subscription/connector.

Step 5 — Validate: confirm the blocker is cleared

1. Run `TBMMDC-Validate.ps1`. It re-checks the affected scope: plan **On**, coverage reporting, connector **connected**, and assessments/recommendations present — pass/fail per link.
2. Re-run `TBMMDC-Coverage.ps1` to confirm the coverage gap closed.
3. Run `TBMMDC-Report.ps1` for the before/after evidence (including Secure Score movement where relevant).
4. Complete the **Testing Guide** for the blocker class before expanding.

Step 6 — Expand: resume the stalled rollout

1. With the affected scope green, enable the plan / provisioning / connector across the remaining subscriptions and connectors in waves (change-approved).
2. Re-run `TBMMDC-Validate.ps1` after each wave and compare with `TBMMDC-Report.ps1`.
3. If a subscription re-introduces the symptom, return to Step 1 for that scope — policy or provider registration can differ per subscription.

Step 7 — Escalate (only if it is a service issue, not your config)

1. If the plan is set On, provisioning/agentless is enabled, the connector role is correct, and the policy is assigned, but the blocker persists, it may be a Microsoft service issue.
2. Run `TBMMDC-SupportBundle.ps1` to assemble `MDC Support Bundle – <case> – Escalate` — the discovery snapshot, the coverage analysis, the affected scope, and timestamps.
3. Open a Microsoft support case, attach the bundle, and reference the escalation criteria in the Troubleshooting Guide. *Incident-response and cost decisions remain the customer's; this is a service-support escalation.*

Order of operations

`Assess (discovery + coverage) → identify earliest broken link → apply the one change-approved fix → re-test the affected scope → validate working → resume rollout → escalate with a bundle only if it is a service issue`

5. Testing Guide — Technical Blocker Mitigation: Defender for Cloud

Run these checks after applying a fix on the affected scope (`MDC Blocker Triage – Affected Scope – Diagnose`) and again after each expansion wave. Each test has a **pass/fail gate**; do not resume the wider rollout until the affected scope passes the test for its blocker class. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Blocker pinpointed (baseline captured)

Procedure: Run `TBMMDC-Discovery.ps1` and confirm `MDC-blocker-discovery.json` has plan states, provisioning settings, connectors, and policy/assessment presence populated; run `TBMMDC-Coverage.ps1` for the coverage baseline. **Pass:** A single earliest broken link is identifiable; the affected scope is documented. **Fail:** Snapshot empty or access-denied — fix read access (Requirements 2.1) and re-run.

Test 2 — Defender plan enabled

Procedure: Run `TBMMDC-Validate.ps1`; in **Defender for Cloud** → **Environment settings** → **Defender plans** confirm the required plan shows **On** for the affected subscription. **Pass:** The required plan(s) show **On** (Standard). **Fail:** Plan Free / will not stay on — see Troubleshooting *Plan will not enable / reverts to Free*.

Test 3 — Agent / agentless coverage

Procedure: Run `TBMMDC-Coverage.ps1`; confirm the affected machines are covered (agent/extension provisioned or agentless scanning enabled) with no unexplained gaps. **Pass:** Covered resources report data; the coverage gap is closed. **Fail:** Machines still report no data — see Troubleshooting *Agent / agentless coverage gap*.

Test 4 — AWS / GCP connector connected

Procedure: Run `TBMMDC-Validate.ps1`; in **Environment settings** confirm the AWS/GCP connector shows **connected / syncing**. **Pass:** The connector is connected and resources appear in scope. **Fail:** Connector not connected / not syncing — see Troubleshooting *AWS/GCP connector failure*.

Test 5 — Recommendations / assessments populate

Procedure: Run `TBMMDC-Validate.ps1`; confirm assessments/recommendations exist for the affected scope and Secure Score returns a value. **Pass:** Assessments populate and Secure Score is present. **Fail:** Empty Secure Score / no assessments — see Troubleshooting *Recommendations not populating*.

Test 6 — Policy assignment present

Procedure: Run `TBMMDC-Discovery.ps1`; confirm the **Microsoft cloud security benchmark** policy assignment exists and is evaluating at the affected scope. **Pass:** The benchmark policy is assigned and evaluating. **Fail:** Missing/failing assignment — see Troubleshooting *Recommendations not populating*.

Test 7 — Before/after evidence produced

Procedure: Run `TBMMDC-Report.ps1` and confirm it emits a before/after comparison (including Secure Score movement where relevant) from the Step 1 snapshot versus current state. **Pass:** The report shows the blocker's link moved from broken to cleared. **Fail:** No change captured — re-run discovery to refresh the baseline.

Test 8 — Escalation bundle valid (only if escalating)

Procedure: If the blocker persists after a correct fix, run `TBMMDC-SupportBundle.ps1` and confirm `MDC Support Bundle – <case> – Escalate` contains the discovery snapshot, coverage analysis, affected scope, and timestamps. **Pass:** A complete, attachable bundle exists. **Fail:** Bundle incomplete — re-run discovery / coverage first, then rebuild.

Exit gate

The affected scope passes the tests for its blocker class **before** you resume the wider rollout (Implementation Step 6). Re-run Tests 2–5 and 7 after each expansion wave and compare against the saved baseline.

6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Cloud

The four blocker classes in **symptom** → **cause** → **fix** form, plus escalation. Run `TBMMDC-Discovery.ps1` and `TBMMDC-Coverage.ps1` first — most causes surface there. This *is* the troubleshooting solution, so this guide is the core reference.

Plan will not enable / reverts to Free

Symptom: A Defender plan for a subscription in `MDC Blocker Triage – Affected Scope – Diagnose` will not turn **On**, or reverts to **Free** after saving. **Cause:** An **Azure Policy / deny assignment** or management-group setting overrides the plan, a required **resource provider** is not registered, or the account lacks the rights to change pricing. **Fix:** In **Environment settings** → **Defender plans**, set the plan **On** and save. If it reverts, check for a policy/deny assignment or MG setting forcing Free, and register the required resource providers. Confirm you have Security Admin/Owner-delegated rights. *Enabling a paid plan has cost impact — change-approved.* Re-run `TBMMDC-Discovery.ps1`.

Agent / agentless coverage gap

Symptom: The plan is **On**, but machines report **no data** and appear as coverage gaps. **Cause:** **Auto-provisioning** of the agent/extension is off, the agent's **Log Analytics workspace** is unreachable/unlinked, or **agentless scanning** is not enabled. **Fix:** Run `TBMMDC-Coverage.ps1` to separate an agent-provisioning gap from an agentless gap. Turn on auto-provisioning of the required agent/extension (or relink the workspace per `global-requirements-defender.md` G7), or enable agentless scanning in the plan settings. Allow provisioning time, then re-run `TBMMDC-Coverage.ps1`.

AWS/GCP connector failure

Symptom: A multicloud connector never finishes onboarding or shows **not connected / not syncing**; AWS/GCP resources are unassessed. **Cause:** The **CloudFormation role** (AWS) or **service account / Terraform** (GCP) has the wrong permissions, onboarding did not complete, or credentials rotated. **Fix:** In **Environment settings**, re-run the connector onboarding — correct the AWS role/permissions or the GCP service account and re-apply the CloudFormation/Terraform artifacts. Confirm the connector returns to **connected / syncing**. *Verify current multicloud onboarding requirements.* Re-run `TBMMDC-Validate.ps1`.

Recommendations not populating

Symptom: Secure Score and assessments stay empty even though plans are On and machines are covered. **Cause:** The **Microsoft cloud security benchmark** policy assignment is missing or failing at the scope, resource providers are not registered, or assessments have not had time to evaluate. **Fix:** Confirm the benchmark policy assignment exists and is evaluating at the affected scope (`TBMMDC-Discovery.ps1` reports it); register required resource providers; trigger/await a compliance evaluation and allow assessments to populate (hours). Re-run `TBMMDC-Validate.ps1`.

Script cannot authenticate / read

Symptom: `TBMMDC-*.ps1` fails on `Connect-AzAccount`, or `Get-AzSecurityPricing` returns access-denied. **Cause:** The Az modules are missing, the wrong tenant/subscription context is selected, or the account lacks Security Reader / Reader. **Fix:** `Install-Module Az`; `Connect-AzAccount` and `Set-AzContext` to the correct tenant/subscription; confirm Security Reader / Reader. Re-run.

When to escalate to Microsoft support

Escalate — and attach `MDC Support Bundle – <case> – Escalate` — when **all** of the following hold:

- The required plan is set **On** (and not overridden by policy/MG) with providers registered.
- Auto-provisioning / agentless scanning is enabled and the workspace is reachable.
- The AWS/GCP connector role/permissions are correct.
- The benchmark policy is assigned and evaluating.
- The correct documented fix was applied and warm-up time elapsed, yet the blocker persists.

In that case the root cause is likely a **service-side issue**, not your configuration. Run `TBMMDC-SupportBundle.ps1`, open the case, and attach the bundle. *Incident-response and cost decisions remain the customer's; this is a service-support escalation.*

7. Best Practices — Technical Blocker Mitigation: Defender for Cloud

Field-proven guidance for diagnosing a stalled Defender for Cloud rollout cleanly, fixing the *right* thing, and respecting the cost impact of paid plans.

Diagnose before you enable anything paid

• Run `TBMMDC-Discovery.ps1` and `TBMMDC-Coverage.ps1` **first**. Read the signal chain top-down; the earliest broken link is the blocker. Enabling a paid plan has **cost impact** — never flip plans on a hunch.

• A "plan is On but nothing is protected" symptom is almost always a **coverage** (agent/agentless) gap, not a plan problem — confirm coverage before touching the plan.

Change one thing, on the affected scope, change-approved, then verify

• Apply exactly **one** documented fix, to the subscription/connector in `MDC Blocker Triage – Affected Scope – Diagnose`, change-approved, then run `TBMMDC-Validate.ps1`. One change at a time keeps cause and effect clear.

• Never resume the estate-wide rollout until the affected scope passes its test gate.

Watch for plans that silently revert

• A plan that will not stay **On** is often overridden by an **Azure Policy / deny assignment** or a management-group setting, or blocked by an unregistered **resource provider**. Check those before assuming the plan toggle is broken.

Coverage: know agent vs agentless

• Distinguish an **agent-provisioning** gap (auto-provisioning off / workspace unreachable) from an **agentless** gap (scanning not enabled). They look the same ("no data") but have different fixes — `TBMMDC-Coverage.ps1` separates them.

Multicloud is a permissions problem until proven otherwise

• AWS/GCP connector failures almost always trace to the **CloudFormation role** (AWS) or **service account / Terraform** (GCP). Re-run onboarding with the correct permissions rather than deleting and guessing. *Verify current multicloud requirements.*

Let plans, coverage, connectors, and assessments warm up

• Plan enablement, provisioning, connector sync, and assessment evaluation populate over **hours**. Give a fix time before calling it a failure, then re-validate.

Keep licensing, plan, and cost facts current

• Plan names, pricing, and capabilities **change**. Before enabling anything, *verify current plan names/pricing in your tenant* — and the enable decision, with its cost, is the customer's.

Escalate with evidence, not a description

• If the blocker survives a correct fix, attach `MDC Support Bundle – <case> – Escalate` (discovery + coverage) to the case. A complete bundle resolves cases far faster than a written

symptom.

Where to go next

Once the rollout is unblocked, natural next steps in the catalog:

- **Defender for Cloud — CSPM & Secure Score Remediation** to move from "onboarded" to "hardened" and drive the score up.
- **Defender XDR Deployment & Unified Onboarding** to correlate cloud-workload alerts with endpoint, identity, and email signal in one portal.
- The **Sentinel** SOC/XDR integration to stream Defender for Cloud alerts into a SIEM for long-term hunting and SOAR.

8. Deliverables Summary — Technical Blocker Mitigation: Defender for Cloud

Documentation

Document	Purpose
Executive Summary	The blockers, the diagnostic-first approach, the outcome, the boundary, effort
1. Solution Overview	The four Defender for Cloud blocker classes, the signal chain (plan → coverage → connectors → policy/assessment), where each breaks
2. Requirements Guide	Diagnostic-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	The diagnostic loop — assess, change-approved fix, re-test the affected scope, validate, resume, escalate
5. Testing Guide	Eight pass/fail tests, one per link in the signal chain
6. Troubleshooting Guide	Every blocker class in symptom → cause → fix, plus escalation criteria
7. Best Practices	Diagnose-first discipline, cost awareness, and where to go next

PowerShell scripts (Az PowerShell — Az.Security / Az.Resources — all read-only)

Script	What it does
`TBMMDC-Discovery.ps1`	Read-only signal-chain snapshot per subscription — Defender plan states (`Get-AzSecurityPricing`), auto-provisioning/agent settings, AWS/GCP security connectors and status, and Microsoft cloud security benchmark policy/assessment presence
`TBMMDC-Coverage.ps1`	Read-only agent / agentless coverage analysis — which resources are covered vs gapped, separating an agent-provisioning gap from an agentless-scanning gap
`TBMMDC-Validate.ps1`	Read-only pass/fail per link for the affected scope — plan On, coverage reporting, connector connected, and assessments/recommendations present
`TBMMDC-Report.ps1`	Read-only before/after status report (including Secure Score movement) from the discovery baseline versus current state — the evidence the blocker cleared
`TBMMDC-SupportBundle.ps1`	Read-only assembly of the support-ready bundle (discovery snapshot, coverage analysis, affected scope, timestamps) for a Microsoft support case

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **read-only PowerShell scripts** in a single ZIP.
- A repeatable diagnostic process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pinpoint the blocker, apply one change-approved fix, re-test the affected scope, confirm working, resume the rollout.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A previously stuck Defender for Cloud rollout moving again — the blocker diagnosed against the signal chain, the targeted documented fix applied and verified on the affected scope, and a support-ready bundle ready if the root cause turns out to be a Microsoft service issue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TBMMDC-Coverage.ps1

Purpose: A "plan is On but nothing is protected" blocker is a coverage gap. This script reports, for the affected subscriptions, the VM/compute inventory and whether machines are instrumented — the Log Analytics / Azure Monitor agent extension and the Defender extension — separating an AGENT-PROVISIONING gap (auto-provisioning off / workspace unreachable) from an AGENTLESS gap. Makes NO changes. HOW TO RUN Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Coverage.ps1 -SubscriptionIds "sub1,sub2" Requires Az.Accounts, Az.Compute, Az.Security. Read-only (Reader). EXPECTED RESULTS A per-subscription coverage summary (machines total vs instrumented) and a flagged gap list, written to MDC-coverage.json. The gap type points at the matching fix. .NOTES White-label. Read-only — enumerates resources/extensions only; changes nothing. Extension names/agents evolve (AMA/MMA, agentless) — verify the current coverage model in your tenant. > [CmdletBinding()] param([string]$SubscriptionIds = "", [string]$OutputPath = ".\MDC-coverage.json")

How to run: Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Coverage.ps1 -SubscriptionIds "sub1,sub2" Requires Az.Accounts, Az.Compute, Az.Security. Read-only (Reader). EXPECTED RESULTS A per-subscription coverage summary (machines total vs instrumented) and a flagged gap list, written to MDC-coverage.json. The gap type points at the matching fix. .NOTES White-label. Read-only — enumerates resources/extensions only; changes nothing. Extension names/agents evolve (AMA/MMA, agentless) — verify the current coverage model in your tenant. > [CmdletBinding()] param([string]$SubscriptionIds = "", [string]$OutputPath = ".\MDC-coverage.json")

TBMMDC-Discovery.ps1

Purpose: Read-only discovery for the "Technical Blocker Mitigation — Defender for Cloud" solution. For each subscription in scope, captures the signal chain: Defender plan states (Get-AzSecurityPricing = On/Free per plan), auto-provisioning/agent settings, AWS/GCP security connectors and their status (ARM REST), and whether the Microsoft cloud security benchmark policy is assigned with assessments present. Makes NO changes. HOW TO RUN Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Discovery.ps1 -SubscriptionIds "sub1,sub2" (Omit -SubscriptionIds to use the current context subscription.) Requires Az.Accounts, Az.Security, Az.Resources. Read-only (Security Reader / Reader). EXPECTED RESULTS A JSON snapshot (MDC-blocker-discovery.json) plus a console summary. Read it top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Uses Get-* / ARM GET only; changes no plan, agent, connector, or policy. Plan names/pricing evolve — verify current availability in your tenant. > [CmdletBinding()] param([string]$SubscriptionIds = "", [string]$OutputPath = ".\MDC-blocker-discovery.json")

How to run: Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Discovery.ps1 -SubscriptionIds "sub1,sub2" (Omit -SubscriptionIds to use the current context subscription.) Requires Az.Accounts, Az.Security, Az.Resources. Read-only (Security Reader / Reader). EXPECTED RESULTS A JSON snapshot (MDC-blocker-discovery.json) plus a console summary. Read it top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Uses Get-* / ARM GET only; changes no plan, agent, connector, or policy. Plan names/pricing evolve —

verify current availability in your tenant. > [CmdletBinding()] param([string]$SubscriptionIds = "", [string]$OutputPath = ".\MDC-blocker-discovery.json")

TBMMDC-Report.ps1

Purpose: Read-only reporting for the "Technical Blocker Mitigation — Defender for Cloud" solution. Compares the discovery baseline (MDC-blocker-discovery.json) with a fresh live read of the same signal-chain links per subscription (plan tiers, assessment count, Secure Score) and prints a before/after summary — the evidence the blocker moved from broken to cleared. Makes NO changes. HOW TO RUN Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Report.ps1 -Baseline .\MDC-blocker-discovery.json Requires Az.Accounts, Az.Security. Read-only. EXPECTED RESULTS A per-subscription baseline vs current table and an overall CLEARED / STILL-BLOCKED verdict, written to MDC-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDC-blocker-discovery.json", [string]$OutputPath = ".\MDC-blocker-report.json")

How to run: Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Report.ps1 -Baseline .\MDC-blocker-discovery.json Requires Az.Accounts, Az.Security. Read-only. EXPECTED RESULTS A per-subscription baseline vs current table and an overall CLEARED / STILL-BLOCKED verdict, written to MDC-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDC-blocker-discovery.json", [string]$OutputPath = ".\MDC-blocker-report.json")

TBMMDC-SupportBundle.ps1

Purpose: Read-only packaging for the "Technical Blocker Mitigation — Defender for Cloud" solution. When the plan is On (not overridden), provisioning/agentless is enabled, the connector role is correct, and the benchmark policy is assigned, yet the blocker persists (likely a Microsoft service issue), this assembles the bundle 'MDC Support Bundle - <case> - Escalate': the discovery snapshot, the coverage analysis, the affected scope, tenant/subscription context, and timestamps. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDC-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedScope "sub1,aws-connector-prod" Point it at the artifacts you already produced: .\TBMMDC-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDC-blocker-discovery.json -Coverage .\MDC-coverage.json EXPECTED RESULTS A single folder 'MDC-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains subscription/cloud-account identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$AffectedScope = "", [string]$Discovery = ".\MDC-blocker-discovery.json", [string]$Coverage = ".\MDC-coverage.json", [string]$Report = ".\MDC-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDC-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedScope "sub1,aws-connector-prod" Point it at the artifacts you already produced: .\TBMMDC-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDC-blocker-discovery.json -Coverage .\MDC-coverage.json EXPECTED RESULTS A single folder 'MDC-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains subscription/cloud-account identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$AffectedScope = "",

[string]$Discovery = ".\MDC-blocker-discovery.json", [string]$Coverage = ".\MDC-coverage.json", [string]$Report = ".\MDC-blocker-report.json")

TBMMDC-Validate.ps1

Purpose: Read-only validation for the "Technical Blocker Mitigation — Defender for Cloud" solution. For each subscription in scope, re-checks the signal-chain links and returns pass/fail: plan On (Standard), assessments/recommendations present, Secure Score returns a value, and (if present) security connectors exist. Makes NO changes. HOW TO RUN Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Validate.ps1 -SubscriptionIds "sub1,sub2" Requires Az.Accounts, Az.Security. Read-only (Security Reader / Reader). EXPECTED RESULTS PASS/FAIL lines per link. FAIL points you to the matching Troubleshooting section. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$SubscriptionIds = "")

How to run: Connect-AzAccount ; then: powershell -ExecutionPolicy Bypass -File .\TBMMDC-Validate.ps1 -SubscriptionIds "sub1,sub2" Requires Az.Accounts, Az.Security. Read-only (Security Reader / Reader). EXPECTED RESULTS PASS/FAIL lines per link. FAIL points you to the matching Troubleshooting section. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$SubscriptionIds = "")